

Ordo — what to build next

Eight tasks in order, each with the technical work it requires. Ordered by risk retired per day, not by interest.

#	Task	Depends on
0	Verify order capture against a live database	Docker restored
1	Measure reply latency (50 real replies)	Task 0
2	Two safe latency fixes	Task 1
3	Variants and stock-accurate answers	—
4	Finish the order loop: Today, notify, stock check	Tasks 0, 3
5	Per-conversation memory	—
6	Images in chat	—
7	Rule-based fast path	Task 1 data

Task 0 is first because the order-capture code is written and type-checked but has never run against Postgres or a real conversation. Nothing built on top of it can be trusted until it is proven.

Task 0 — Verify order capture end to end

The multi-turn order flow exists in code with 23 passing unit tests, but the database half has never executed. Prove it before adding anything.

Technical work

- Restore Docker, then `docker compose up -d` and `npm run migrate` (11 migrations, none new).
- Drive a real conversation through WhatsApp: item, name, phone, address, then “ow hari”.
- Assert in the database, not in the UI, at each turn.

Checks

```
-- draft accumulates, does not reset
SELECT state, customer_name, phone, address,
       (SELECT count(*) FROM order_items oi WHERE oi.order_id = o.id) AS items
FROM orders o WHERE conversation_id = '...';

-- after the customer confirms
-- state must be PENDING_CONFIRMATION, never CONFIRMED (section 15)
SELECT o.state, e.from_state, e.to_state, e.actor
FROM orders o JOIN order_events e ON e.order_id = o.id;
```

Watch for

- A field given at turn 2 still present at turn 5 — the merge must not erase.
- Exactly one DRAFT order per conversation, not one per message.
- Total computed from the database price, including the delivery fee.

Files

`src/orders/persist.ts`, `src/orders/draft.ts`, `src/worker/pipeline.ts` (step 9)

Task 1 — Measure reply latency

One live sample came in at 12.4s against an 8s p95 gate — but that sample included a failed model call and two retries. Optimising against one worst case puts effort in the wrong layer.

Technical work

Everything needed is already recorded. Collect 50 real replies, then read the distribution rather than the mean — the mean hides the tail that the gate is actually about.

```
SELECT
  count(*)                                AS calls,
  percentile_cont(0.50) WITHIN GROUP (ORDER BY latency_ms) AS p50,
  percentile_cont(0.95) WITHIN GROUP (ORDER BY latency_ms) AS p95,
  max(latency_ms)                          AS worst,
  avg(input_tokens)::int AS in_tok,
  avg(output_tokens)::int AS out_tok,
  model
FROM ai_decisions
WHERE created_at > now() - interval '7 days'
GROUP BY model;
```

Decides

If p50 is already under 5s, Task 2 is the whole of the latency work and Task 7 can be dropped. If p50 is 7s+, the model call itself is the problem and model choice has to be revisited with an eval set.

Task 2 — Two safe latency fixes

Both are hours of work, carry no quality risk, and are measurable immediately. Do them before anything that trades quality for speed.

2a — Parallelise retrieval

`retrieveContext` makes **6 sequential database round trips**: products, FAQs, history, contact, recent failures, draft order. They are independent of one another.

```
// src/ai/retrieval.ts – before
const products = await db.query(...); // each waits for the previous
const faqs     = await db.query(...);
const history  = await db.query(...);

// after
const [products, faqs, history, contact, failures, draft] =
  await Promise.all([ ... ]);
```

Expected saving 100–400ms depending on database latency. On a remote database it is worth considerably more.

One caveat: all six run inside the same `withTenant` transaction, which uses a single pooled connection. `Promise.all` on one client serialises anyway. Either issue them on separate scoped connections, or accept the smaller win of combining the cheap ones into a single query with CTEs.

2b — Lower `maxOutputTokens`

Currently 1024. Replies are one or two sentences — roughly 40–120 tokens. The model is not billed for unused capacity, but generation time scales with what it actually produces, and a high ceiling gives it room to ramble.

```
// src/ai/gemini-client.ts
maxOutputTokens: 1024, -> maxOutputTokens: 256,
```

256 leaves headroom for an order read-back, which is the longest reply the assistant ever writes. Verify against the longest real reply in `outbound_messages` before lowering further.

Acceptance

- p50 and p95 re-measured with the Task 1 query and recorded.
- No change in intent accuracy or reply quality on the same messages.

Task 3 — Variants and stock-accurate answers

This is the highest-risk correctness gap in the system. There is no `product_variants` table, so “size 40 thiyenawada?” is answered from the product-level status and is confidently wrong whenever one size is out.

A wrong stock answer produces an order the seller cannot fulfil. That costs them a customer and costs us the trust the product is built on — which is why this comes before images.

Schema — migration 0013

```
CREATE TABLE product_variants (  
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  tenant_id   uuid NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,  
  product_id  uuid NOT NULL REFERENCES products(id) ON DELETE CASCADE,  
  label       text NOT NULL,          -- 'Size 40', 'Blue', 'Blue / 40'  
  sku         text,  
  price       numeric(12,2),          -- NULL = inherit the product price  
  stock_status stock_status NOT NULL DEFAULT 'AVAILABLE',  
  quantity    integer,                -- optional; status is what we answer with  
  is_active   boolean NOT NULL DEFAULT true,  
  UNIQUE (tenant_id, product_id, label)  
);  
-- RLS: same tenant_isolation policy as every other table.
```

Technical work

- **Variants are optional.** A product with no variant rows answers exactly as it does today. A shop selling one-size items is never made to model them.
- **Reuse the Singlish matcher.** Run variant labels through `generateAliasKeys` so “size 40”, “40 size” and “size40” fold to one key. This inherits every future Singlish improvement instead of adding a second matcher.
- **Retrieval returns variants** with the product, and the prompt renders them with their own stock status.
- **Price still comes from the database** — variant price if set, otherwise the product's. The model never sees either.
- **CSV import learns variants.** A 'Size' or 'Variant' column becomes variant rows rather than five near-duplicate products.

Answering rules — in code, not in the prompt

Situation	Behaviour
No variants exist	Answer from product <code>stock_status</code> (unchanged)
Variant named, in stock	Answer yes, with that variant's price
Variant named, out of stock	Say so; offer the sizes that ARE in stock
Variant named, not in catalogue	Hand off. Never guess
No variant named, several exist	Ask which, before quoting anything
Any <code>OUT_OF_STOCK</code> variant	Never enters a draft order

Acceptance

- “size 40 thiyenawada?” answers from the variant row.
- An out-of-stock size is never sold, and the reply offers what is available.
- A shop with no variants sees no behaviour change at all.

Task 4 — Finish the order loop

The order reaches `PENDING_CONFIRMATION`. What is missing is everything that makes the seller notice and act on it.

4a — Today tiles

The dashboard already counts orders. What it does not show is the number the seller has to act on.

Tile	Query basis
Waiting for your confirmation	<code>state = 'PENDING_CONFIRMATION'</code>
Captured by the assistant today	<code>conversation_id IS NOT NULL AND created_at::date = current_date</code>
Drafts in progress	<code>state = 'DRAFT'</code>

The third is worth building even though nobody asks for it: a rising count of abandoned drafts is the earliest signal that customers are giving up partway through the order flow, and it is invisible in every other metric.

4b — Notification

`notifySeller()` in `pipeline.ts` is still a stub. A confirmed order nobody is told about sits until the seller happens to look.

- Minimum: unread badge already works — surface `PENDING_CONFIRMATION` count on the Orders tab the same way.
- Better: browser push via the PWA, which needs a service worker and VAPID keys.
- Do NOT send the seller a WhatsApp message per order — at 30 orders a day that is 30 notifications and they will mute the number.

4c — Stock check at confirmation

Nothing currently prevents an order for an item that went out of stock during the conversation. Re-check every line at the moment the customer confirms; if anything is unavailable, do not place the order — hand off to the seller with the reason. Depends on Task 3.

4d — Delivery fee by city

`delivery_rules` has both `colombo_rate` and `outstation_rate`; `defaultDeliveryFee()` always uses the Colombo one. Select on the captured city, and fall back to outstation when the city is unknown — undercharging delivery is the seller's money, not ours.

Task 5 — Per-conversation memory

Each buyer's history must inform the next message. “Eka kiyada?” after three messages about a saree has to resolve to that saree.

The gap

`retrieval.ts` passes the last 10 raw messages and nothing else. A 30-message conversation loses its beginning entirely, and the model re-infers what “eka” means every turn — sometimes differently from the turn before, which reads to the customer as the shop forgetting.

5a — Conversation memory table (migration 0014)

```
CREATE TABLE conversation_memory (  
  conversation_id uuid NOT NULL REFERENCES conversations(id) ON DELETE CASCADE,  
  key              text NOT NULL,    -- 'last_product', 'delivery_city', ...  
  value            text NOT NULL,  
  product_id       uuid REFERENCES products(id) ON DELETE SET NULL,  
  updated_at       timestampz NOT NULL DEFAULT now(),  
  PRIMARY KEY (conversation_id, key)  
);
```

Written by **code**, not by the model: whichever product retrieval matched with the highest score becomes `last_product`. No extra model call, no extra cost.

5b — State the referent explicitly

```
<conversation_memory>  
  Currently discussing: Silk Saree (id 7c9e...)  
  Customer asked about: price, delivery to Kadawatha  
</conversation_memory>
```

Telling the model what “eka” refers to is far more reliable than hoping it re-derives the same answer from prose each turn.

5c — Rolling summary

Above 20 turns, replace the oldest half with a one-paragraph summary from a single cheap model call, stored on the conversation and refreshed every 10 further turns. Cost is amortised — roughly one extra call per 10 messages.

```
ALTER TABLE conversations  
  ADD COLUMN summary text,  
  ADD COLUMN summary_through_message_id uuid,  
  ADD COLUMN summarised_at timestampz;
```

Acceptance

- “Silk saree kiyada?” → “delivery kiyada?” → “mata eka one” drafts an order for the SAREE.
- A 40-message conversation still answers correctly about message 2.
- Tokens per call stop growing with conversation length.

Task 6 — Images in chat

A customer sends a photo and asks “mekka thiyenawada?”. Today the image is detected, then discarded.

The gap

`session.ts` records `mediaType` and sets `mediaUrl: null`. Nothing is downloaded and nothing is looked at; the message reaches the inbox as an unhandled item.

6a — Download and store

```
// src/channels/waweb/media.ts
downloadMediaMessage(raw, 'buffer', {})
-> media/{tenantId}/{messageId}.{ext}
-> messages.media_url = relative path
-> messages.media_sha256 = hash // customers resend the same photo
```

6b — Vision in the same call

Gemini accepts images natively — a real advantage of the provider already chosen. Attach the image to the call that already classifies intent, so an image message costs one model call, not two.

6c — Match through the existing matcher

Ask the model to **describe** the item and pick from the candidate list. Never ask it to produce a product id from an image. Run the description through the same alias and trigram matcher used for text, so image matching inherits every Singlish improvement for free.

Cost: an image adds roughly 250–1,000 input tokens. Downscale to 512px on the long edge before sending — product identification does not need more, and it keeps the added cost near Rs 0.10 rather than Rs 0.40.

Also required

- Media is untrusted input exactly as text is. The injection posture does not relax because the content is a picture.
- Never serve media from a public path without an authorisation check — one tenant's customer photos must not be reachable by another.
- A retention policy belongs in this task, not later: delete media after N days, keep the message row. Storage otherwise grows without bound.

Acceptance

- A photo of a stocked product is answered by name, with price and stock from the database.
- A photo of something not stocked hands off rather than guessing.
- The image is visible in the seller inbox.

Task 7 — Rule-based fast path

A greeting does not need a model call. Removing the model from the cheapest messages is the largest single latency and cost win available.

Technical work

A narrow exact-match table checked before the model call, in `pipeline.ts` after the spend guards:

```
hi · hello · hey · ayubowan · ayubowan!    -> greeting reply
sthuthi · thanks · thank you · bohoma sthuthi -> acknowledgement
ok · okay · hari                           -> no reply at all
```

Expected to remove the model from 15–25% of messages, saving the whole call (2–3s) and its cost on each.

Keep it deliberately narrow: exact matches on greetings and thanks only, never anything that could be a question. The moment the rule table starts answering product questions it has become a second, worse assistant that nobody is measuring.

Why last

It needs real intent data from Task 1 to be drawn safely. Guessing which messages are trivial, before seeing what customers actually send, is how a real question ends up answered with “Ayubowan!”.

Acceptance

- Greeting-only messages reply in under 500ms with no model call.
- No message containing a question mark, a product name or a number is ever matched by the fast path.
- `ai_decisions` shows the drop in call volume.

Release gates — unchanged

These do not move for any task above.

Gate	Threshold	Where measured
Price hallucinations	Zero. One blocks release	<code>ai_decisions.gate_triggered</code>
Intent accuracy	85%	Needs a labelled set — see below
Reply latency	5s median, 8s p95	<code>ai_decisions.latency_ms</code>

The labelled set does not exist yet, and building it is worth more than any single task above. 100–200 real Sri Lankan customer messages with the correct intent and the correct reply attached. Without it, every change to the prompt or the model is an opinion; with it, each one becomes a measurement. It also supplies the 12–20 real few-shot examples the prompt still carries as placeholders.

Written against the codebase as it stands: 11 migrations, 43 source files, 13 test files. Every gap listed was verified in the code, not assumed.